

Week 2

Week 2: The Dialogue Engine - Roles, the Loop, and Budgets

KIUA2003 — Applikasjon av kunstig intelligens / maskinl ring II

Build a reusable `DialogueEngine` with correct role-mapping (`view_for`), three stop conditions, and structured JSON transcripts.

CONTENTS

Reading	Lecture Notes: The Dialogue Engine
Document	Lecture Slides - Week 2
Document	Weekly Handout - Week 2 (PDF)
Reading	This Week's Code
Reading	Thought for 4s: The Engine in Plain Language
Reading	Code Explanation: The Dialogue Engine
Quiz	Week 2 Quiz: Dialogue Engine
Discussion	Week 2 Discussion: Role-Mapping Bugs & Stop Conditions
Reading	Supplementary Resources

READING

Lecture Notes: The Dialogue Engine

Week 2 - The Dialogue Engine: roles, the loop, and budgets

This week's idea: turn last week's ad-hoc ping-pong into a real, reusable **engine** -

one class that takes two agents and a budget, runs them to a stop condition, and saves a clean transcript. The conceptual heart is **role-mapping**: making each agent correctly see the **other** agent as the user and **itself** as the assistant.

By the end you have a `DialogueEngine` driven by a config file, enforcing three independent stop conditions, writing a structured transcript.

Lecture

1. The role-mapping problem (the key insight)

The conversation is **symmetric in the world** but **asymmetric for each model**. When it's Agent B's turn, B must receive a message list where:

- B's own previous lines are `assistant`,
- A's lines are `user`,
- B's system prompt is the `system` message.

So there is **no single shared message list** - each agent has its **own view** of the same events. The engine stores one neutral transcript and **renders** a per-agent view on demand:

```
def view_for(agent, transcript):
    msgs = [{"role": "system", "content": agent.system_prompt}]
    for entry in transcript:
        role = "assistant" if entry.speaker == agent.name else "user"
        msgs.append({"role": role, "content": entry.content})    return msgs
```

Draw this on the board with a 3-message example for both agents side by side - it's the "aha" of the week.

2. The orchestration loop

Strip an agent conversation to its skeleton:

```
while not budget.exhausted():
    speaker = next_speaker(transcript)          # simplest: alternate A, B
    view = view_for(speaker, transcript)
    reply = client.chat(speaker.model, view, temperature=speaker.temperature)
    transcript.append(Entry(speaker.name, reply.text, reply.tokens, reply.seconds))
    budget.record(turns=1, tokens=reply.tokens, seconds=reply.seconds)
    if goal_reached(transcript):                budget.stop("goal_reached")
```

That's the entire engine. Everything else is configuration and bookkeeping.

3. Turn-taking

Simplest is strict alternation A, B, A, B. Mention richer options (a moderator agent decides who's next - a Week-4 stretch goal) but keep alternation for now.

4. Termination & budgets

Four ways a dialogue ends - implement at least the first three:

- **Max turns** - count of messages.
- **Token budget** - running sum of generated tokens.
- **Wall-clock** - elapsed seconds.
- **Goal/stop condition** - a domain check (price agreed; judge says done; a stop-phrase emitted).

The engine should record **which** condition fired (stop_reason) - you'll report it.

5. Configuration over hard-coding

Models, persona prompts, temperatures, and budgets belong in a **config file** (YAML/JSON), not scattered in code. This makes Week 3's experiments (vary one thing, re-run) trivial.

6. The transcript as data

Save runs as **JSON**, one record per message: `speaker, role, content, tokens, seconds, turn\_index`, plus a run header (models, temps, budgets, stop_reason, totals). A good transcript is the thing you analyse in Weeks 3–4 - design it deliberately.

Lab

Exercise A - implement `view\_for` (30 min)

In `engine.py`, complete the `view_for` TODO. Test it with a hand-made 3-entry transcript: print the view for Agent A and for Agent B and confirm the roles flip correctly. ****This is the make-or-break exercise**** - don't move on until both views are right.

Exercise B - the loop (35 min)

Complete `DialogueEngine.run()`: the `while not budget.exhausted()` loop, calling `view_for`, the client, appending entries, and recording the budget. Run it in mock mode for 6 turns. Then run it for real.

Exercise C - three stop conditions (25 min)

Configure a run that stops on **max turns**. Then one that stops on **token budget** (set it low). Then one that stops on **wall-clock** (set it to a couple of seconds). Confirm `engine.stop_reason` reports the right cause each time.

Exercise D - save & inspect transcripts (30 min)

Write each run to `transcripts/.json` with per-message metrics and a run header. Open one and check it has everything you'd need to write your report without re-running.

Project task → Milestone M2

Submit a `DialogueEngine` that:

1. Reads a **config file** (models, personas, temperatures, budgets) - nothing key hard-coded.
2. Uses **correct role-mapping** (`view_for`) so each agent sees the other as user.
3. Enforces **three independent stop conditions** (turns, tokens, time) and records which fired.
4. Saves a **structured JSON transcript** with per-message tokens & time and a run header.

Checklist

- [] `view_for` flips roles correctly (demonstrated)
- [] Loop runs your scenario end-to-end
- [] All three budgets independently stop a run; `stop_reason` correct
- [] Config file drives the run
- [] Transcript JSON has per-message tokens/seconds + run header
- [] Runs cleanly in both `--mock` and real mode

Common pitfalls

- **Role-mapping wrong** → agents talk past each other or repeat themselves; the model thinks it said things it didn't. Re-check `view_for` first whenever dialogue looks insane.
- **Mutating one shared message list** instead of rendering per-agent views - the classic bug.

- **Counting only turns** and forgetting tokens/time - a single 3 000-token turn shows why you need all three.
- **Transcript saved as plain text** - you'll regret it in Week 3 when you need the numbers.

DOCUMENT

Lecture Slides - Week 2

Attached file: slides-week2.pdf

DOCUMENT

Weekly Handout - Week 2 (PDF)

Attached file: week2-dialogue-engine.pdf

READING

This Week's Code

This Week's Code

You are building the **DialogueEngine** -- the reusable class that replaces last week's ad-hoc `ping_pong.py`.

What to edit

Open `engine.py`. There are two TODOs:

TODO 1: `view_for(agent, transcript)` (around line 29)

Build the message list from one agent's point of view:

- First message: `{"role": "system", "content": agent.system_prompt}`
- For each entry in the transcript: "assistant" if this agent spoke it, "user" if the other agent spoke it

Test it by hand before moving on: create a 3-entry transcript, print the view for Agent A and Agent B, confirm the roles are mirror images.

TODO 2: `DialogueEngine.run()` (around line 69)

The loop skeleton is in the docstring. Fill in the body:

```
while not self.budget.exhausted():
    speaker = self.next_speaker()
    messages = self.manage_context(view_for(speaker, self.transcript))
    reply = self.client.chat(speaker.model, messages, speaker.temperature)
    # append an Entry(...) to self.transcript      # record the turn in the budget
    # check goal_reached
```

What to run

```
# Step 1: test with mock (free, instant)
python run.py --config configs/debate.yaml --mock # Step 2: test with real model
python run.py --config configs/debate.yaml # Step 3: check the saved transcript
cat transcripts/debate.json | python -m json.tool
```

What success looks like

- `view_for` flips roles correctly (each agent sees the other as user)
- The loop runs your scenario end-to-end and stops on a budget limit
- `transcripts/debate.json` contains per-message tokens, seconds, and a run header with `stop_reason`
- Config file drives everything (no hard-coded models, prompts, or budgets)

Files you touch this week

File	What you do
<code>`engine.py`</code>	Implement <code>`view_for()`</code> and <code>`run()`</code>
<code>`configs/debate.yaml`</code>	Copy and edit for your scenario (change personas, topic, budgets)

All other files (`budget.py`, `llm_client.py`, `agents.py`, `run.py`) are unchanged from Week 1.

READING

Thought for 4s: The Engine in Plain Language

Thought for 4s: The Engine in Plain Language

Let me walk through it piece by piece, because the engine is really just two ideas glued together.

Idea 1: there is one conversation, but each agent gets its own "view" of it

The engine stores a single neutral list of who-said-what:

```
1. A: "I offer 80." 2. B: "Too low: 120." 3. A: "Meet at 95?"
```

But an LLM API doesn't accept "A said / B said". It only understands three labels: `system` (your persona), `assistant` (things you said before), and `user` (things said to you). So when it's A's turn, A's own lines must be labelled `assistant` and B's lines `user`. When it's B's turn, the labels are the opposite. That's what `view_for` does:

```
def view_for(agent, transcript):    msgs = [{"role": "system", "content": agent.system_
prompt}]    # this agent's persona first    for entry in transcript:
        # walk the neutral record in order        role = "assistant" if entry
        .speaker == agent.name else "user"    # "did I say this, or did they?"
        msgs.append({"role": role, "content": entry.content})    return msgs
```

It never changes the content or the order. It only decides, for each line, "was that me (`assistant`) or the other one (`user`)?" Called with agent A you get one list; called with agent B on the same transcript you get a different list. That's why the two agents can't share one message list: the correct labels are different for each of them.

Idea 2: the loop just repeats "pick speaker -> build their view -> ask the model -> record the reply"

```

while not budget.exhausted():
    t = next_speaker(transcript)
    e A, B, A, B) view = view_for(speaker, transcript)
    # keep going until some limit is hit
    # whose turn? (simplest: alternate)
    # build the message list from THEIR point of view
    reply = client.chat(speaker.model, view, temperature=speaker.temperature)
    # call the LLM
    transcript.append(Entry(speaker.name, reply.text, reply.tokens, reply.seconds))
    # store the reply neutrally
    budget.record(turns=1, tokens=reply.tokens, seconds=reply.seconds)
    # spend some budget
    if goal_reached(transcript):
        # e.g. a price was agreed
        budget.stop("goal_reached")

```

Trace one iteration. Transcript is empty. `next_speaker` says A. `view_for(A, [])` gives just A's system prompt. The model replies "I offer 80." That gets appended as `Entry("A", "I offer 80.", ...)`. Next iteration: `next_speaker` says B. `view_for(B, transcript)` gives B's persona plus user: "I offer 80.". B replies "Too low: 120", appended as spoken by B. Next iteration: A's view is now A's persona, assistant: "I offer 80.", user: "Too low: 120.", and so on.

Notice the loop itself contains no intelligence. It's plain Python that decides who speaks, what they see, and when to stop. The handout calls that the **orchestrator**. The LLMs only ever produce the reply text.

The three parts you have to fill in around the loop

- `budget.exhausted()` becomes true when any of three counters passes its limit: number of turns, total tokens, or elapsed seconds. Whichever one fires gets written to `stop_reason`, so later you can tell "this run hit max_turns" apart from "this run agreed a price."
- `next_speaker` is deliberately one tiny function so you can swap strict alternation for something fancier (a moderator) in Week 4 without touching the rest.
- Everything you'd want to change between experiments (models, personas, temperatures, the three limits) is read from a **config file** instead of being typed into the code.

Every `Entry` and the final `stop_reason` get dumped to JSON so Week 3's analysis can be done from the file alone.

Debugging tip: if the agents start repeating themselves or arguing their opponent's side, the labels in `view_for` are almost certainly flipped. The model is being told it said things it didn't.

READING

Code Explanation: The Dialogue Engine

Code Walkthrough: The Dialogue Engine

This document explains the code you are building and using in Week 2. Read it alongside the source files.

1. Why `engine.py` replaces `ping_pong.py`

Last week's `ping_pong.py` worked, but it was a one-off script. The entire conversation was stuffed into a single user message as flat text, so the model had to guess which lines were its own and which belonged to the other agent. That is fragile: the model sometimes mixes up speakers, and there is no clean way to add features (context management, goal detection, saving) without rewriting the script

each time.

`engine.py` solves both problems. It separates the **data** (who said what) from the **loop** (whose turn it is, when to stop), and it uses proper chat roles (`system`, `user`, `assistant`) so the model sees the conversation the way it was trained to.

2. ``Entry``: one message in the transcript

```
@dataclass class Entry:      speaker: str      content: str      prompt_tokens: int
      completion_tokens: int  seconds: float      turn_index: int
```

Compare this with Week 1's transcript, which was a list of `(name, text)` tuples. `Entry` adds four fields: the two token counts, the wall-clock time, and the position in the sequence. These numbers come straight from `ChatResponse` (see Week 1's `llm_client.py` explanation) and are stored so that `save` can write them into the JSON transcript. You will need them in Week 4 when you build your results table.

`turn_index` is the entry's position in the overall conversation. The first message ever spoken has `turn_index=0`, the second has `turn_index=1`, and so on. It is set when the entry is created inside `run()`, using `len(self.transcript)` at that moment. This means the index is always correct even if you later filter or slice the transcript.

3. ``view_for``: building what the model sees

```
def view_for(agent, transcript):    ...
```

This is the function you implement. Its job: take a flat transcript (a list of `Entry` objects) and return a list of message dictionaries that look like a normal chat conversation from one agent's point of view.

Why it matters

Chat models are trained on conversations with three roles:

Role	Meaning
<code>`system`</code>	Background instructions the model should follow
<code>`assistant`</code>	Things the model itself said previously
<code>`user`</code>	Things the other party said

If you send the entire transcript as a single `user` message (like `ping_pong.py` did), the model has no structural way to tell its own lines from the other agent's lines. It has to parse names out of the text, which is unreliable. By using `assistant` for the agent's own past replies and `user` for the other agent's replies, you give the model the same structure it was trained on.

The mirror property

The same transcript produces two different views. If Alice spoke first:

```

Transcript: [Alice: "Hi", Bob: "Hello", Alice: "How are you?"] Alice's view:
system:      "You are Alice."
assistant:   "Hi"                (Alice said this -- it is her own line)
user:        "Hello"              (Bob said this -- it is the other party)
assistant:   "How are you?"      (Alice again) Bob's view:  system:      "You are Bob."
user:        "Hi"                (Alice said this -- it is the other party)
assistant:   "Hello"              (Bob said this -- it is his own line)
user:        "How are you?"      (Alice again)

```

Notice that assistant and user are swapped. The two views are mirror images of each other. This is the entire point of the function.

The empty-transcript case

When the transcript is empty, the model would receive a system prompt and nothing else. Some models handle this fine; others get confused without a user message. The function adds a starter message like "You speak first." with role user so there is always something for the model to respond to. This starter only appears on the very first turn.

4. `DialogueEngine.\_\_init\_\_`: the constructor

```

def __init__(self, agents, client, budget, goal_reached=None,
              manage_context=None):
    self.agents = agents
    self.client = client
    self.budget = budget
    self.transcript = []
    self.goal_reached = goal_reached or (lambda t: False)
    self.manage_context = manage_context or (lambda messages: messages)

```

The constructor stores everything the engine needs and sets two hooks to safe defaults.

The `or` pattern for optional callbacks

`goal_reached or (lambda t: False)` means: if the caller passed a function, use it; if they passed `None` (or nothing), use a lambda that always returns `False`. This is a common Python idiom for optional callbacks. The effect is that `run()` can always call `self.goal_reached(self.transcript)` without checking whether a function was provided. The same pattern applies to `manage_context`: the default lambda returns the messages untouched, so the engine works without context management until you add it in Week 3.

Why hooks and not subclasses

An alternative design would be to make `DialogueEngine` a base class and override methods in subclasses. Hooks (functions you pass in) are simpler: you do not need to define a new class just to change one behaviour. The config-driven approach in `run.py` also benefits, because a function is easier to attach from a config than a subclass.

5. `next\_speaker`: whose turn is it?

```

def next_speaker(self):
    return self.agents[len(self.transcript) % len(self.agents)]

```

The modulo operator (%) cycles through the agent list. With two agents, even indices pick agent 0 and odd indices pick agent 1, which is the same alternation `ping_pong.py` used. But because it uses `len(self.agents)` instead of hard-coding 2, the same logic works for three or more agents.

6. `run()`: the main loop

```

def run(self):
    while not self.budget.exhausted():
        speaker = self.next_speaker()
        messages = self.manage_context(view_for(speaker, self.transcript))
        reply = self.client.chat(speaker.model, messages, speaker.temperature)
        entry = Entry(
            speaker=speaker.name,
            content=reply.text,
            prompt_tokens=reply.prompt_tokens,
            completion_tokens=reply.completion_tokens,
            seconds=reply.seconds,
            turn_index=len(self.transcript),
        )
        self.transcript.append(entry)
        self.budget.record(turns=1, tokens=reply.tokens)
        if self.goal_reached(self.transcript):
            self.budget.stop("goal_reached")
            return self.transcript

```

This is the function you implement. Each line maps to a concept:

Line	What it does
<code>`while not self.budget.exhausted()`</code>	The budget decides when to stop (see Week 1: <code>`budget.py`</code>)
<code>`speaker = self.next_speaker()`</code>	Pick the next agent by cycling through the list
<code>`messages = self.manage_context(...)`</code>	Build the view, then optionally compress it (Week 3)
<code>`reply = self.client.chat(...)`</code>	Call the model and get back a <code>`ChatResponse`</code>
<code>`entry = Entry(...)`</code>	Wrap the reply and its metadata into a transcript entry
<code>`self.transcript.append(entry)`</code>	Add it to the running transcript
<code>`self.budget.record(...)`</code>	Tell the budget about the cost so it can check limits
<code>`if self.goal_reached(...)`</code>	Optional early stop if the scenario's goal is met
<code>`return self.transcript`</code>	Hand the full conversation back to the caller

Why ``turn_index=len(self.transcript)`` works

At the moment you create the entry, `self.transcript` has not yet received the new entry. So `len(self.transcript)` is the index the new entry will occupy once appended. Entry 0 is created when the list is empty (`len == 0`), entry 1 when the list has one item (`len == 1`), and so on.

Why ``reply.tokens`` and not ``reply.completion_tokens``

`reply.tokens` is a property that returns `prompt_tokens + completion_tokens` (see the `ChatResponse` class from Week 1). The budget tracks total tokens consumed, not just the ones the model generated, because prompt tokens also cost money on paid APIs and contribute to rate limits.

7. ``save``: writing the transcript to disk

```

def save(self, path, meta=None):
    import os
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    record = {
        "meta": meta or {},
        "agents": [{"name": a.name, "model": a.model,
                    "temperature": a.temperature}
                  for a in self.agents],
        "budget": self.budget.summary(),
        "totals": { ... },
        "messages": [asdict(e) for e in self.transcript],
    }
    with open(path, "w") as f:
        json.dump(record, f, indent=2)
    return path

```

``os.makedirs`` with ``exist_ok=True``

`os.makedirs` creates the directory (and any missing parents). `exist_ok=True` means "do not raise an error if the directory already exists". The `or "."` handles the case where `path` is a bare filename with no directory part: `os.path.dirname("run.json")` returns `""`, and `makedirs("")` would fail, so `or "."` substitutes the current directory.

``asdict``

`asdict` is from the `dataclasses` module. It converts an `Entry` (a `dataclass` instance) into a plain dictionary `{"speaker": ..., "content": ..., ...}`. This is necessary because `json.dump` can serialise dictionaries and lists but not arbitrary Python objects.

The ``totals`` block

The totals aggregate numbers across all entries:

```
"totals": {      "turns": len(self.transcript),
  "prompt_tokens": sum(e.prompt_tokens for e in self.transcript),
  "completion_tokens": sum(e.completion_tokens for e in self.transcript),
  "seconds": round(sum(e.seconds for e in self.transcript), 3), }
```

`sum(e.prompt_tokens for e in self.transcript)` is a generator expression: it walks through every entry, pulls out `prompt_tokens`, and adds them up. You will use these totals in Week 4 to build your results table without having to re-compute anything.

8. Connecting it: ``run.py`` and YAML configs

You do not change `run.py` this week, but understanding how it uses the engine helps:

```
agents = [Agent(**a) for a in cfg["agents"]] budget = Budget(**cfg.get("budget", {}))
client = make_client(mock=mock) engine = DialogueEngine(agents, client, budget)
transcript = engine.run()
engine.save(out, meta={"topic": cfg.get("topic"), "config": path})
```

The `**` operator unpacks a dictionary into keyword arguments. `Agent(**a)` is equivalent to `Agent(name=a["name"], system_prompt=a["system_prompt"], ...)`. This is why the keys in your YAML file must match the field names in the `dataclass` exactly: a typo like ``system-prompt`` instead of ``system_prom`

Summary of what is new this week

Concept	Where	Why it matters
Proper chat roles (<code>`system`/`user`/`assistant`</code>)	<code>`view_for`</code>	Models see structure, not flat text
<code>Dataclass</code> with metadata fields	<code>`Entry`</code>	Every message carries its cost and position
Hook-based design (optional callbacks)	<code>`DialogueEngine.__init__`</code>	Extensible without subclassing
Modulo cycling	<code>`next_speaker`</code>	Works for any number of agents
JSON transcript with totals	<code>`save`</code>	Self-contained record for later analysis

QUIZ

Week 2 Quiz: Dialogue Engine

1. When it is Agent B's turn, Agent A's earlier messages must appear in B's view with role:

- A. ``user``
- B. ``assistant``
- C. ``system``
- D. ``agent_a``

2. Why can the engine not keep one message list and send it to both agents?

- A. The list would grow too large for the context window.
- B. The ``user`/`assistant`` labels are perspective-dependent; each agent needs the roles flipped its own way.
- C. Ollama only allows one system prompt per process.
- D. It could, but per-agent views are faster.

3. The purpose of recording ``stop_reason`` is:

- A. To let the model know it should stop talking.
- B. To restart the run from where it stopped.
- C. To report and debug why each run ended: goal reached, or which budget fired.
- D. To satisfy the config file schema.

4. Your agents behave, but one turn suddenly generates 4,000 tokens. Which stop condition is designed to catch this?

- A. Max turns
- B. Token budget
- C. Goal condition
- D. Strict alternation

5. Which of the four ways a dialogue ends is the "happy path"?

- A. ``max_turns``
- B. ``max_tokens``
- C. ``max_seconds``
- D. The goal condition (deal agreed, judge satisfied, stop-flag emitted)

6. "Strict alternation" (A, B, A, B, ...) is an example of:

- A. A message protocol
- B. A turn-taking policy
- C. A stop condition
- D. Role-mapping

7. Which of these does not belong in the config file?

- A. The two persona system prompts
- B. The temperature of each agent
- C. The implementation of ``view_for``
- D. The three budget limits

8. A good transcript's run header contains:

- A. Only the final message, to save space.
- B. The Python source of the engine.
- C. Models, temperatures, budgets, ``stop_reason``, and run totals.
- D. The student's name and grade.

9. For each entry, the neutral transcript stores:

- A. A ready-made ``user`/`assistant`` label, so views can be copied out directly.

- B. A neutral speaker name; `user`/`assistant` labels exist only inside a rendered per-agent view.
- C. Only the message text; who said it is inferred from position.
- D. The rendered views for both agents, stored side by side.

10. Adding a moderator agent that decides who speaks next requires:

- A. Rewriting the engine's main loop.
- B. Replacing only the `next\_speaker` hook.
- C. Switching to a framework such as AutoGen.
- D. A second, separate transcript for the moderator.

11. Your max-turns cap is correctly enforced. Why are the token and wall-clock budgets still needed?

- A. Turn counting is blind to turn size and duration: twelve 4,000-token turns, or one turn that hangs for ten minutes, both pass a turns-only check.
- B. They are not needed; they exist only for symmetry.
- C. Ollama requires all three limits to be set on every call.
- D. A max-turns cap cannot be enforced reliably in Python.

12. Saving transcripts as structured JSON with per-message metrics mainly pays off when:

- A. The dialogue needs to look professional in the report appendix.
- B. Week 3's experiments need turns, tokens, seconds, and `stop\_reason` without re-running anything.
- C. The transcript must be smaller than a plain-text log.
- D. The judge agent can only read JSON.

13. The lab's make-or-break test for `view\_for`: build a small hand-made transcript and print the rendered views for both agents. What exactly must you see?

- A. The two views are identical, proving the transcript is shared.
- B. The roles flip between the two views (each agent's own lines are `assistant`, the other's are `user`) while content and order stay identical.
- C. Each view contains only that agent's own messages.
- D. Both views start with the same system prompt.

14. Mid-run, your agents start repeating themselves and "answering their own questions". The first thing to check is:

- A. The temperature: it is probably too high.
- B. The model: 3B models cannot hold a dialogue.
- C. `view\_for`: broken role-mapping makes an agent see its own words attributed to the other side, so it responds to itself.
- D. The token budget: the history was probably truncated.

15. `Context:** ``` role = "user" if entry.speaker == agent.name else "assistant" ```` A student's `view\_for` contains the line above. Which corrected line fixes the role-mapping?**

- A. `role = "assistant" if entry.speaker == agent.name else "user"`
- B. `role = "system" if entry.speaker == agent.name else "user"`
- C. `role = "user" if entry.speaker == agent.name else "system"`
- D. The line is already correct.

DISCUSSION

Week 2 Discussion: Role-Mapping Bugs & Stop Conditions

The role-mapping problem (view_for) is the 'aha' of this week. Share: What happened when your role-mapping was wrong? Which stop condition fired first in your runs, and was it the one you expected?

READING

Supplementary Resources

Week 2 - Supplementary Resources

Core references

- **Anthropic - "Building Effective Agents"** - The canonical reference for agent architecture. Defines workflows vs agents and why simple beats complex.

[anthropic.com/research/building-effective-agents](https://www.anthropic.com/research/building-effective-agents)

- **Lilian Weng - "LLM Powered Autonomous Agents"** - Decomposes agents into Planning + Memory + Tool Use. The most-cited agent survey blog post.

lilianweng.github.io/posts/2023-06-23-agent

- **Steve Kinney - "The Anatomy of an Agent Loop"** - Shows that every framework (Claude Code, Codex, LangGraph, smolagents) converges on the same while-loop you are building.

stevekinney.com/writing/agent-loops

Going further

- **AI Builder Club - "How to Build an AI Agent from Scratch"** - A complete agent in ~60 lines of Python, no framework. Good reference for your own engine.

[aibuilderclub.com/blog/how-to-build-ai-agent-from-scratch](https://www.aibuilderclub.com/blog/how-to-build-ai-agent-from-scratch)

- **nerdai/llm-agents-from-scratch** (GitHub) - Companion code for the Manning book "Build a Multi-Agent System (From Scratch)". Builds from a single agent through tool-calling to multi-agent systems.

github.com/nerdai/llm-agents-from-scratch

- **Lay4U/build-your-own-ai-agent** (GitHub) - 12 practical projects mastering AI agents from scratch. No frameworks, just Python and an LLM.

github.com/Lay4U/build-your-own-ai-agent

- **Wu et al. - "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation"** (2023) - Defines the conversable-agent abstraction and orchestration patterns you are building by hand. Read this now to see what your engine will look like as a framework; re-read in Week 4 for the comparison. arXiv:2308.08155

arxiv.org/abs/2308.08155

- **Li et al. - "CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society"** (NeurIPS 2023) - The paper that introduced inception prompting, where each agent sees the other as "user". This is exactly the `view_for` concept you implement this week. arXiv:2303.17760

arxiv.org/abs/2303.17760